

10 · Views and Indexes

Why this matters in interviews. "What is a view? Can you insert into it?", "What does `WITH CHECK OPTION` do?", "How does an index speed up a query — and when does it not?", "Why isn't my index being used?" These two objects are the gateway to performance and security questions.

In this chapter: views (create, replace, drop, updatable, `WITH CHECK OPTION`) · indexes (how they work, kinds, composite order, when they help and when they don't).

Part A — Views

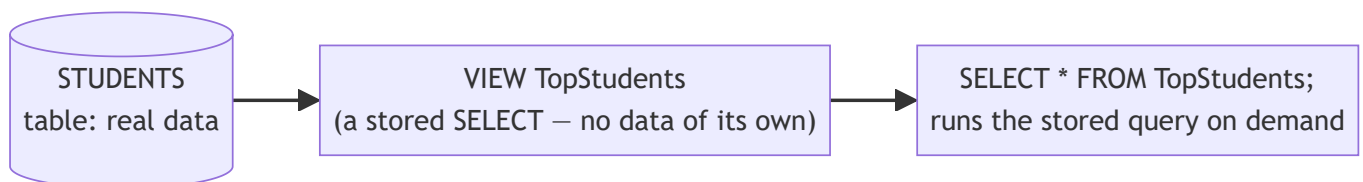
10.1 What is a view?

A **view** is a **virtual table based on the result of a `SELECT` query**. It **doesn't store data itself** — it stores the *query*, and shows data from one or more actual tables every time you read it.

Think of a view as:

- a **saved SQL query** you can treat like a table,
- a **window** into your data.

Views are useful for **simplifying complex queries**, **improving security** (expose only some columns/rows) and **reusing logic** (write the tricky join once).



10.2 Creating a view

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Take a `students` table:

students

student_id	name	marks	grade
1	Asha	92	A
2	Bala	78	B
3	Chitra	85	A

student_id	name	marks	grade
4	Dinesh	64	C
5	Esha	91	A

A view that shows only students with marks above 80:

```
CREATE VIEW TopStudents AS
SELECT name, marks
FROM students
WHERE marks > 80;
```

SELECT * FROM TopStudents;

name	marks
Asha	92
Esha	91
Chitra	85

Now you can **query it like a table** — filter it, join it, sort it. Because it's live, a student whose marks are later raised above 80 appears in it automatically.

10.3 Modifying and dropping a view

```
-- change the definition: CREATE OR REPLACE
CREATE OR REPLACE VIEW TopStudents AS
SELECT name, marks, grade
FROM students
WHERE marks > 90;

-- remove the view (the underlying table is untouched)
DROP VIEW TopStudents;
```

```
CREATE OR REPLACE VIEW TopStudents AS
SELECT name, marks, grade
FROM students
WHERE marks > 90;
```

Table after

name	marks	grade
Asha	92	A
Esha	91	A

CREATE OR REPLACE swaps the definition without dropping the view (and without losing grants made on it). Dropping a view **never** deletes data from the base table.

10.4 A view over a join

The everyday use — hide a three-table join behind one name:

```
CREATE OR REPLACE VIEW emp_dept_vw AS
SELECT e.employee_id, e.first_name, d.department_name
FROM   employees e
JOIN    departments d ON e.department_id = d.department_id;

SELECT * FROM emp_dept_vw WHERE department_name = 'Sales';
```

10.5 Can you change data through a view?

Sometimes — a view is **updatable** only if the database can trace each row back to exactly one base-table row.

A view is generally updatable if it...	It is read-only (or needs an INSTEAD OF trigger) if it contains...
selects from one table	DISTINCT
has no aggregates	GROUP BY , HAVING , aggregate functions
keeps the base table's key	set operators (UNION , MINUS ...), ROWNUM
	most joins (only the "key-preserved" table can be changed)

For non-updatable views Oracle lets you define an **INSTEAD OF trigger** that decides what an **INSERT** / **UPDATE** / **DELETE** on the view should do to the base tables (Chapter 15). **WITH READ ONLY** makes a view explicitly read-only.

WITH CHECK OPTION — the "bouncer at the club"

WITH CHECK OPTION makes sure that any **INSERT** or **UPDATE** done **through the view must still satisfy the view's WHERE condition**. The **WHERE** is the *dress code*; **WITH CHECK OPTION** stops rows from sneaking in wearing something that breaks it.

Without it:

```
CREATE VIEW high_salary_employees AS
SELECT employee_id, employee_name, salary
FROM   employees
WHERE  salary > 50000;

INSERT INTO high_salary_employees (employee_id, employee_name, salary)
VALUES (6, 'John', 30000);
```

The insert **succeeds** — but the new row **doesn't appear in the view**, because 30,000 fails **salary > 50000** . The row went into the base table and immediately vanished from sight.

With it:

```
CREATE OR REPLACE VIEW high_salary_employees AS
SELECT employee_id, employee_name, salary
FROM employees
WHERE salary > 50000
WITH CHECK OPTION;

INSERT INTO high_salary_employees VALUES (6, 'John', 30000);
-- ORA-01402: view WITH CHECK OPTION where-clause violation
```

The insert is **rejected**. The same applies to an `UPDATE` that would push a row out of the view (`SET salary = 20000`). Use `WITH CHECK OPTION CONSTRAINT name` to name the check.

10.6 Views vs tables, and materialized views

	Table	View	Materialized view (Oracle)
Stores data	✓	✗ (stores the query)	✓ (a stored snapshot of the result)
Always current	✓	✓ (computed on each read)	✗ until refreshed
Speed	direct	as fast as its query	fast reads — no recomputation
Typical use	the data itself	security, simplification	reports, aggregates, data warehouses

Find views in the dictionary: `SELECT view_name, text FROM user_views;`

Part B — Indexes

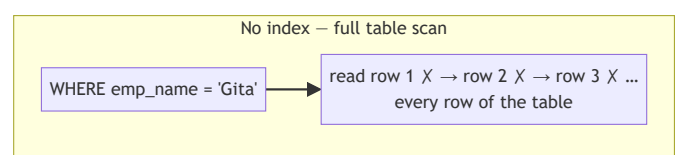
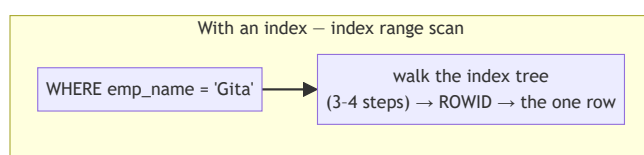
10.7 What is an index?

An **index** is a database object that **improves the speed of data retrieval** (like `SELECT`) by letting the engine **locate rows quickly instead of scanning the whole table**.

Think of the index of a book. Instead of flipping through every page looking for "Normalisation", you look it up in the index and jump straight to the page number.

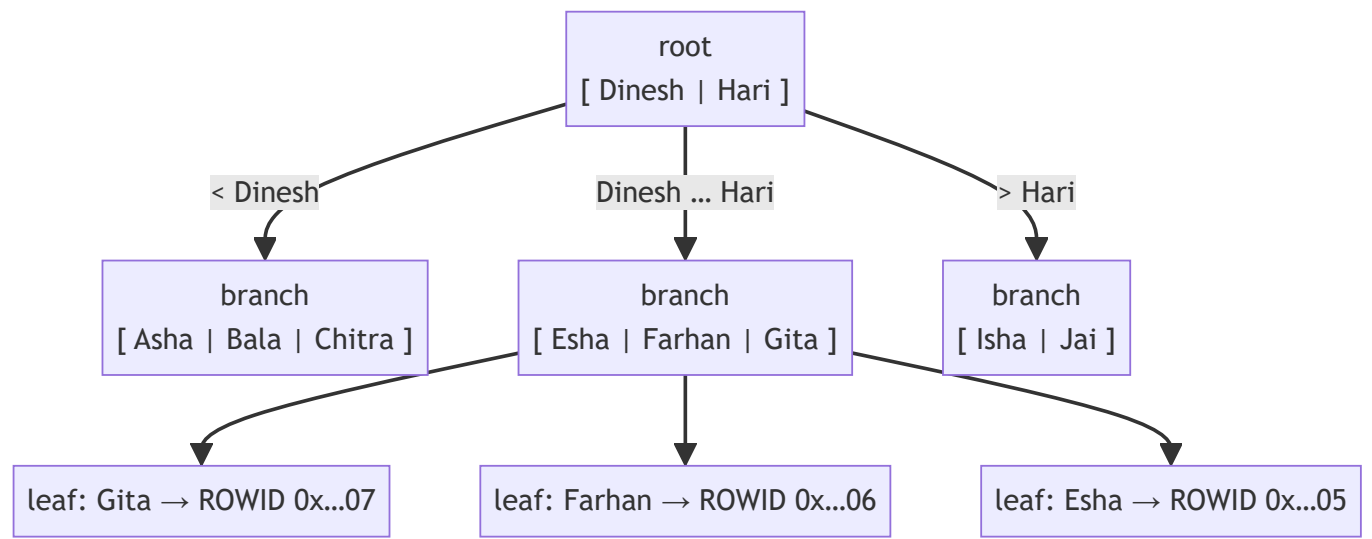
Why use an index?

- Speeds up search queries — `WHERE` , `JOIN` , `ORDER BY` ...
- Helps performance on large datasets.
- Is **created automatically on** `PRIMARY KEY` and `UNIQUE` constraints.



10.8 How it works: the B-tree

Most indexes (Oracle's default) are **B-tree** structures — a balanced tree of sorted keys. The database walks from the root, choosing the branch by comparing values, until it reaches a leaf that holds the key and a **ROWID** pointing to the row. A tree with millions of keys is only 3–4 levels deep, which is why a lookup takes a handful of reads instead of millions.



Because the keys are **sorted**, a B-tree is good for **equality, range queries** (**BETWEEN** , **<** , **>**), **ORDER BY** and **prefix LIKE 'Ga%'** . Other implementations:

Index kind	Good for	Notes
B-tree	equality, ranges, sorting	the default nearly everywhere
Hash	exact-match lookups only (not ranges)	PostgreSQL/MySQL-MEMORY; Oracle has hash <i>clusters</i> , not hash indexes
Bitmap <i>(Oracle)</i>	columns with few distinct values (gender, status), read-mostly / warehouse	poor for frequent updates
Function-based	queries on an expression	see below

10.9 Creating indexes

1 · Basic index

```
CREATE INDEX idx_lastname
ON Employees (LastName);
```

Speeds up queries like `SELECT * FROM Employees WHERE LastName = 'Sharma';` .

Proof, on our sample data — the **plan** for the same query before and after (SQLite's plan shown; Oracle's **EXPLAIN PLAN** prints `TABLE ACCESS FULL` vs `INDEX RANGE SCAN`):

```
EXPLAIN PLAN FOR SELECT * FROM emp WHERE emp_name = 'Gita';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Result

Operation (SQLite plan)

SCAN emp

```
CREATE INDEX idx_emp_name ON emp (emp_name);
EXPLAIN PLAN FOR SELECT * FROM emp WHERE emp_name = 'Gita';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Result

Operation (SQLite plan)

SEARCH emp USING INDEX idx_emp_name (emp_name=?)

SCAN = the whole table is read; SEARCH ... USING INDEX = the index is used. In Oracle the second plan reads INDEX RANGE SCAN IDX_EMP_NAME followed by TABLE ACCESS BY INDEX ROWID .

2 · Composite (multi-column) index

```
CREATE INDEX idx_name_dept
ON Employees (LastName, DepartmentID);
```

Useful when queries filter on **both columns together**: WHERE LastName = 'Kumar' AND DepartmentID = 3;

Note: the order matters. A composite index is sorted by the **first** column, then the second within it (like a phone book: last name, then first name). It helps only when the query uses the **leading column(s)** — the **leftmost-prefix rule**:

Query on (LastName, DepartmentID)	Uses the index?
WHERE LastName = 'Kumar'	✓ yes (leading column)
WHERE LastName = 'Kumar' AND DepartmentID = 3	✓ yes — both columns
WHERE DepartmentID = 3	✗ normally no — the leading column is missing (see *)
ORDER BY LastName, DepartmentID	✓ can avoid a sort

*Oracle's optimizer may still choose an **index skip scan** when the leading column has very few distinct values.

Rule of thumb for column order: equality columns first, most selective first, range column last.

3 · Unique index

```
CREATE UNIQUE INDEX idx_email ON Users (Email);
```

Ensures **no two users have the same email**. It acts like a **UNIQUE** constraint **with indexing benefits** (the constraint creates such an index for you).

4 · Function-based (expression) index

A plain index on `Email` **cannot** serve `WHERE LOWER>Email) = 'user@example.com'` — the function hides the column. Index the *expression* instead (PostgreSQL, Oracle, MySQL 8):

```
CREATE INDEX idx_lower_email ON Users (LOWER>Email));

SELECT * FROM Users WHERE LOWER>Email) = 'user@example.com'; -- now index-supported, speeds up
case-insensitive queries
```

5 · Dropping an index

```
DROP INDEX idx_lastname; -- Oracle, PostgreSQL
DROP INDEX idx_lastname ON Employees; -- MySQL / SQL Server syntax may vary
```

Dropping an index never touches table data.

10.10 When an index does *not* help — or hurts

Do not index (or expect help):

Situation	Why
Small tables	scanning a few blocks is faster than walking an index
Frequent INSERT / UPDATE / DELETE	every index must be updated as well — writes get slower
Too many indexes	wastes storage and slows down write performance
Low-selectivity columns (a <code>gender</code> column with 2 values, when a query returns half the table)	the optimizer will rightly prefer a full scan

The optimizer skips an index when the query "hides" the column (a *non-sargable* predicate):

Query	Problem	Fix
<code>WHERE UPPER(emp_name) = 'GITA'</code>	function on the indexed column	function-based index, or compare <code>emp_name = 'Gita'</code>
<code>WHERE salary * 12 > 500000</code>	arithmetic on the column	<code>WHERE salary > 500000 / 12</code>
<code>WHERE emp_name LIKE '%ita'</code>	leading wildcard	prefix search <code>'Gi%'</code> , or a text index
<code>WHERE emp_id = '101'</code>	implicit type conversion of the column	compare with the right data type
<code>WHERE comm IS NULL</code>	B-tree indexes don't store all-NULL keys	add a constant column to the index, or use a bitmap/function index

Oracle does not automatically index foreign-key columns. Index the child's FK column (`orders.customer_id`) or joins and parent-row deletes will scan the whole child table (Chapter 03).

Clustered vs non-clustered (a term you will hear)

In **SQL Server** and MySQL **InnoDB** the table itself can be physically stored in the order of a **clustered index** (the primary key) — only one per table. Other indexes are **non-clustered**: a separate structure holding key + pointer. Oracle's ordinary tables are *heap* tables; the closest thing is an **index-organised table (IOT)**.

Looking at what exists

```
SELECT index_name, table_name, uniqueness FROM user_indexes;
SELECT index_name, column_position, column_name FROM user_ind_columns WHERE table_name = 'EMP';
```

Interview corner

Q. What is a view? Does it store data? A virtual table defined by a `SELECT`; it stores only the query, not data.

Q. Can you `INSERT / UPDATE / DELETE` through a view? Yes for simple, single-table views; not for views with `DISTINCT`, `GROUP BY`, aggregates or most joins (unless an `INSTEAD OF` trigger handles it).

Q. What is `WITH CHECK OPTION`? It ensures rows inserted or updated through the view still satisfy the view's `WHERE`; otherwise the DML fails (`ORA-01402`).

Q. View vs materialized view? A view recomputes its query each time; a materialized view stores the result and must be refreshed.

Q. What is an index and how does it work? A separate sorted structure (usually a B-tree) mapping key values to row locations, so the engine finds rows without a full table scan.

Q. Drawbacks of indexes? Extra storage, and slower `INSERT / UPDATE / DELETE` because every index must be maintained.

Q. Composite index (a, b) — does `WHERE b = 1` use it? Generally no — the leftmost-prefix rule; the query must use `a` (Oracle may attempt a skip scan).

Q. Why might the optimizer ignore an index? Function or arithmetic on the column, leading-wildcard `LIKE`, implicit conversion, low selectivity, tiny table, stale statistics.

Q. Primary key vs unique index? A primary key is a constraint (`NOT NULL` + unique identity) that is *implemented* with a unique index; a unique index alone allows NULLs and isn't a key definition.

Q. B-tree vs hash vs bitmap? B-tree: equality + range + sort. Hash: equality only. Bitmap: low-cardinality columns in read-mostly systems.

Q. How do you find out whether a query uses an index? `EXPLAIN PLAN FOR <query>; SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);` — look for `INDEX RANGE SCAN` vs `TABLE ACCESS FULL`.

Quick recap

- **View** = stored `SELECT`, no data; simplify · secure · reuse. `CREATE [OR REPLACE] VIEW ... AS SELECT ..., DROP VIEW`.
- Simple views are updatable; `WITH CHECK OPTION` blocks rows that would leave the view; `WITH READ ONLY` forbids DML.

- **Index** = sorted structure (B-tree) that finds rows without a full scan — created automatically for **PRIMARY KEY** / **UNIQUE** .
- Kinds: **B-tree** (default) · hash · **bitmap** · **function-based** · unique · composite (**leftmost-prefix rule — order matters**).
- Don't index small tables, write-heavy tables, or low-selectivity columns; too many indexes slow writes.
- Indexes are bypassed by functions/arithmetic on the column, leading to **%**, and type conversions.
- Oracle doesn't auto-index foreign keys.